Lab assignment 4: Speech technology

Introduction to programming (LIS020)

Robert Östling
robert@ling.su.se

Amanda Kann
amanda.kann@ling.su.se

Maria Skeppstedt
maria.skeppstedt@ling.su.se

October 19, 2026

Purpose and learning outcomes

In this assignment, you will use the external libraries `numpy`, `librosa` and `python-soundfile` to load speech data with multiple speakers, cut out pauses, detect the mean fundamental frequency (F_0) of each segment, and produce a version of the input data with the speaker order sorted by F_0.

To achieve a given grade for this assignment, you will need to:

- **Pass**: Correctly implement the processing chain described below under *Tasks*, independently designing the code based on the instructions and what you read in the documentation for the external libraries.
- **Pass with distinction**: In addition to the above, follow best practices for all aspects of code documentation, error handling, structure and flexibility.

Submitting your solution

The naming and number of Python files is up to you, but should contribute towards the goals of well-documented code. Submit the Python files to Canvas.

Background

The main purpose of this lab assignment is to practice becoming an independent programmer. Therefore, these instructions are specified at a higher level than previous assignments, intentionally leaving many of the design choices to you.

When grading your assignment, we will assess the following aspects of your code:

- Documentation (docstrings, type annotations, comments, identifier names). Note that basic type annotations are sufficient for `numpy` types, in practice this means that all arrays can be declared to be of the type `numpy.ndarray`.
- Code structure (does the code avoid redundancy, and is the structure appropriate for the task and easy to understand?)
- Code flexibility and reusability (how easy would it be to modify your code or reuse parts of it for other purposes?) Note that the task of this assignment is not very complex, so in practice this criterion boils down to creating maybe one or two functions that could conceivably be useful outside of this particular program. You do not have to create a large amount of functions for the sake of writing functions.
- Error handling. If the user does not specify the correct command-line arguments, if the input data is not of the format expected or can not be read for some reason, your program should give informative error messages to the user rather than crashing.

To pass with distinction, best practices must be followed with respect to the above criteria.

Data

In this lab assignment, you will be working with the following file:
`/home/dsv/robe/lis020/labs/data/seven.wav`

It is also available on the following URL, for your convenience:
`https://people.dsv.su.se/~robe/lis020/data/seven.wav`

The file is compiled form the AudioMNIST database[1], and consists of 60 speakers pronouncing the word "seven" in English.

Tasks

You should write a program that does the following:

1. Read a data file specified by the user. This can be done with `soundfile.read`. Read up on command-line arguments using `sys.argv` in Python if you are not already familiar with this.
2. Cut the recording into the 60 clips of "seven" contained in it. This can be done with `librosa.effects.split`.

[1] `https://github.com/soerenab/AudioMNIST`

3. Compute the F_0 curve of each clip. This can be done with `librosa.pyin`. You can use the value 70 for the `fmin` parameter and 500 for `fmax`. Note also that you need to pass the correct sampling rate that you got from `soundfile.read`.

4. Compute the mean F_0 of each clip, given the F_0 curve. This can be done using `numpy.nanmean`. Note that for a few clips, the F_0 curve could not be computed (it consists of only `nan` values). Discard these clips. The functions `numpy.isnan` and `numpy.all` can be used to check whether a numpy array contains only `nan` values.

5. Sort the remaining clips by mean F_0, lowest to highest.

6. Concatenate the clips, using the `numpy.concatenate` function.

7. Write the resulting data to a wav file specified by the user, with the same sample rate as the input data.

Unless otherwise specified above, you can assume that the default values to the library functions are OK.

One should be able to run your program like this:

```
python3 process.py /home/dsv/robe/lis020/labs/data/seven.wav
↪  ~/public_html/output.wav
```

where the first argument is the input file, and the second one is the output file.

If you do the above, you can easily play the output sound file by writing the following URL into your web browser or media player: `https://people.dsv.su.se/~USERNAME/output.wav` where "USERNAME" is replaced by your Stockholm University username.

Use our virtual environment

Since the libraries are rather large, it would be wasteful if every student in this course created a separate virtual environment. You did this in the previous lab assignment because the library used then, `pyconll`, is much smaller. Here, you can instead use a virtual environment that we have created for you, with the relevant libraries already installed:

```
source /home/dsv/mariask/venv/lab4/bin/activate
```

Sanity check

In order to help you check that your program works as expected, here are the estimated F_0 in Hz (rounded to the nearest integer), in sorted order:

```
[70, 70, 80, 82, 83, 85, 89, 91, 94, 97, 102, 103, 104, 104,
↪  104, 108, 110, 116, 119, 119, 119, 120, 121, 124, 124,
↪  125, 126, 126, 126, 129, 130, 131, 132, 132, 132, 133,
↪  133, 134, 135, 139, 139, 144, 145, 147, 149, 157, 168,
↪  177, 185, 195, 203, 225, 232, 238, 241, 241, 243, 252,
↪  257]
```

There are 59 values here, because `librosa` was unable to determine F_0 for one of the 60 clips, which was discarded according to the specification above.